Advanced Spark SQL in Databricks

Building production-grade pipelines with powerful SQL patterns, window functions, and schema-aware data engineering.

DATA ENGINEERING | SPARK SQL | DATABRICKS

Recap: Where We Left Off

Before diving into advanced territory, let's anchor ourselves in the foundations. Two concepts form the bridge between programmatic and declarative data work in Databricks.

DataFrame ↔️ SQL

Spark DataFrames and SQL are two sides of the same coin. You can seamlessly switch between writing DataFrame transformations and running SQL queries — both compile to the same underlying execution plan.

Temporary Views

Temp views let you register a DataFrame as a named SQL table scoped to your session. This is the gateway from programmatic data wrangling into the SQL world — and a pattern you'll use constantly in real pipelines.

SQL in Real Pipelines

Spark SQL isn't just for ad-hoc querying — it's a first-class citizen in production data pipelines. Understanding where SQL fits in your workflow helps you write cleaner, more maintainable code.

Aggregations

Computing **SUM**, **COUNT**, **AVG**, and grouped metrics across large datasets. SQL makes complex aggregation logic readable and highly optimised by the Catalyst engine.

Reporting

Generating business-ready summaries and denormalised views for downstream consumption — dashboards, BI tools, or analytical notebooks.

Transformations

Reshaping, filtering, joining, and enriching raw data as part of a structured medallion architecture — Bronze → Silver → Gold.

Managing Databases in Databricks

Moving beyond session-scoped temp views means working with **persistent databases**. Databricks uses the Hive Metastore (or Unity Catalog) to track databases and their objects across sessions and users.

1

CREATE DATABASE

Registers a new database in the metastore. Optionally specify a **LOCATION** to control where data files are stored. Without it, Databricks manages the path automatically.

2

USE

Sets the active database context so subsequent queries don't need to qualify every table name with a database prefix. Reduces verbosity in interactive and scripted workflows.

3

SHOW DATABASES

Lists all databases visible in the current metastore. Essential for exploration, debugging, and verifying your environment before running pipeline steps.

Key mindset shift: Stop thinking in session-scoped temp views and start thinking in persistent, shareable database objects. This is the foundation of collaborative, production-grade data engineering.

Tables in Databricks: Managed vs. External

Choosing the right table type is a critical architectural decision. It determines who controls the data lifecycle — Databricks or you.

Managed Tables

Databricks controls both the **metadata and the underlying storage**. When you run `DROP TABLE`, the data files are deleted along with the table definition. Best for ephemeral or intermediate pipeline layers where you trust Databricks to own the lifecycle.

- Storage path managed automatically
- Simple to create — no LOCATION required
- Data deleted on DROP

External Tables

You control the **storage location** (e.g., an S3 or ADLS path). Databricks only manages the metadata. Dropping the table removes the definition but **leaves data intact**. Essential for data shared across multiple systems or tools.

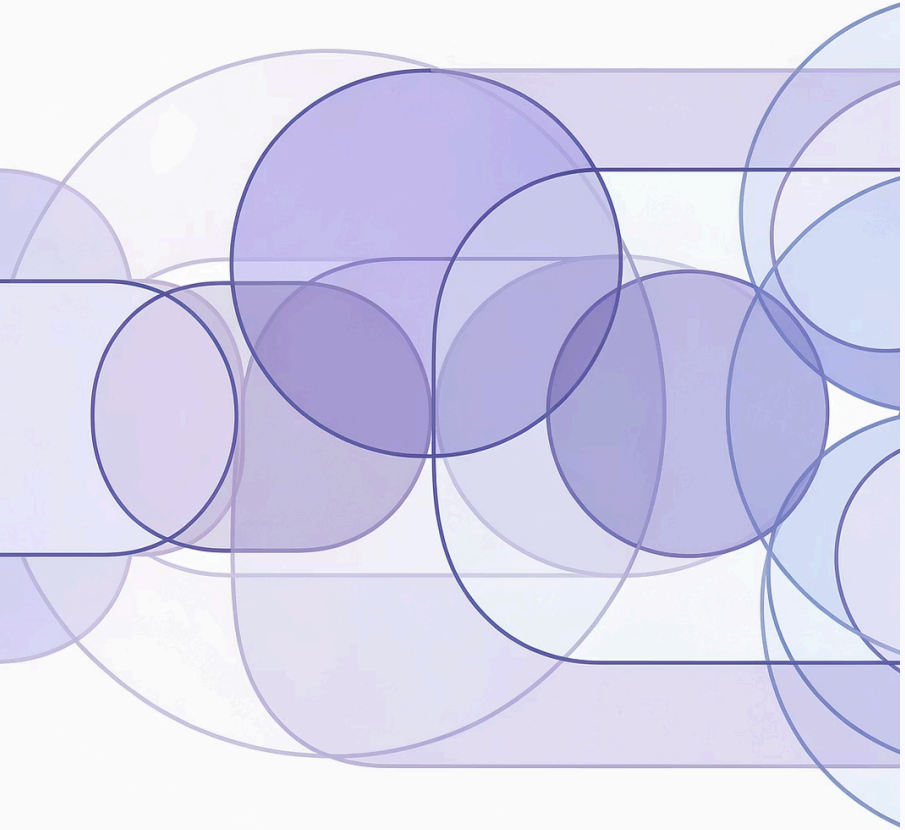
- Requires explicit LOCATION path
- Data persists after DROP TABLE
- Ideal for shared or regulated data

Temporary vs. Global Temporary Views

Not all views are created equal. Understanding scope is **critical in multi-user and multi-notebook environments** — using the wrong view type can lead to data leakage, stale references, or broken pipelines.

View Type	Scope	Best Used For
Temporary View	Current Spark session only. Dropped when session ends.	Single-notebook workflows, quick exploration, isolated transformations.
Global Temporary View	Shared across all notebooks in the same Spark cluster. Lives in `global_temp` database.	Sharing intermediate results across notebooks in a multi-step pipeline on the same cluster.
Persistent View / Table	Available across sessions, clusters, and users via the metastore.	Shared reporting layers, Gold tables, and cross-team data products.

Multi-user tip: In collaborative environments, global temp views can cause subtle bugs if two users share a cluster. Prefer persistent tables or carefully namespaced views for team pipelines.



Advanced SQL Operations

The goal here isn't to memorise syntax — Spark SQL is largely ANSI-compliant and well-documented. The focus is on **recognising which query pattern fits your data problem**.

Joins

INNER — matching rows only. **LEFT** — all left rows, nulls on right where no match. **RIGHT** — the inverse. Choose based on whether unmatched rows carry meaning in your pipeline.

Aggregations

Use GROUP BY with aggregate functions (SUM, COUNT, MAX, AVG). Combine with HAVING to filter aggregated results — a pattern common in reporting and anomaly detection.

Grouping Sets

GROUPING SETS, ROLLUP, and CUBE extend standard GROUP BY to produce multi-level summaries in a single query pass — highly efficient for hierarchical reporting.

Window Functions

Window functions are one of the most powerful — and most misunderstood — tools in SQL. They compute values **across a set of rows related to the current row**, without collapsing the dataset like `GROUP BY` does.

ROW_NUMBER()

Assigns a unique sequential integer to each row within a partition. Used for **deduplication** — keep only the row where `row_num = 1` per key.

RANK() / DENSE_RANK()

Ranks rows within a partition. `RANK` skips numbers on ties; `DENSE_RANK` does not. Essential for **top-N analysis** and leaderboard patterns.

LEAD() / LAG()

Access the value of a row **ahead or behind** the current row within a partition. Invaluable for computing period-over-period changes, time-series deltas, and sequential event analysis.

Common Use Cases

- Deduplicate records keeping the latest version
- Rank customers by revenue within a region
- Calculate day-over-day metric changes
- Identify gaps in sequential event streams

Pattern: Always define `PARTITION BY` and `ORDER BY` carefully — they determine correctness, not just performance.

NULL Handling: A Data Correctness Issue

NULLs are not just inconvenient — they are a **correctness risk**. Silently propagating NULLs through aggregations, joins, and comparisons is one of the most common sources of subtle, hard-to-diagnose data quality bugs in production pipelines.

1

Identify

Use `IS NULL` and `IS NOT NULL` to detect missing values. Never use `= NULL` — this always evaluates to NULL, not TRUE.

2

Replace

Use `COALESCE(col, default)` to substitute the first non-NULL value from a list. A clean, readable pattern for applying fallback values in transformations.

3

Handle in Aggregations

Remember: `COUNT(*)` includes NULLs; `COUNT(col)` does not. `SUM` and `AVG` ignore NULLs — which can silently skew your results.

Best practice: Treat NULL handling as a first-class concern at every layer of your pipeline. Document expected nullability in your schema and enforce it with data quality checks.

SQL Works Only if the Schema is Correct

Next: Schema Design & Enforcement

We've covered the full spectrum of advanced SQL operations — from persistent databases and table types, through window functions and NULL semantics. But all of this SQL power rests on a single foundation: **the schema must be correct**.

A perfectly crafted window function returns garbage if the underlying column types are wrong. A `COALESCE` on a column that should never be NULL signals a schema contract has been broken. In the next section, we go deeper — into schema definition, enforcement, evolution, and the data contracts that make pipelines trustworthy at scale.

What's Next

- Defining schemas explicitly in Databricks
- Schema enforcement vs. schema evolution
- Delta Lake and ACID guarantees
- Data contracts in team pipelines